3.2 相容性 (Compatibility)

本節探討客體程式在虛擬機與原生平台執行行為的一致性。

- **相容性層級**：分為「本質相容性（Intrinsic）」，要求對所有程式 100% 準確模擬；以及「外在相容性（Extrinsic）」，僅針對特定子集（如特定編譯器產生的程式）保證相容。
- **充分相容條件**：相容性不要求每一條指令的執行細節都完全相同，核心要求在於**作業系統控制轉移點（OS Control Transfer Points）**。當控制權轉移至 OS（如系統呼叫、Trap）或從 OS 回復執行時，客體與主體的狀態必須在映射函數下等效。
- **實作相容性限制**：某些硬體實作細節（如指令快取與資料快取的區隔）可能在架構中變得可見，導致如「自我修改代碼」這類行為出現不確定性，增加虛擬化難度。

3.3 狀態映射 (State Mapping)

說明如何將客體系統的資源（暫存器、記憶體）映射到主體平台。

- **暫存器映射**：若主體暫存器數量多於客體，可直接對映；若數量不足，則須在記憶體中維護一個「暫存器內容塊（Register Context Block）」，在模擬模式與 Runtime 模式切換時換入/換出內容。
- **記憶體映射**：
  - **軟體轉換表**：由 Runtime 維護類似分頁表的結構，最靈活但速度慢。
  - **直接映射 (Direct Mapping)**：若主體位址空間足夠大，可將客體位址直接對應到主體位址（零位移或固定位移），效能最佳。
- **位址空間共享**：Runtime 軟體必須與客體應用程式共享同一個主體位址空間，這涉及位址空間大小與分頁大小不匹配的議題。

3.5 指令模擬 (Instruction Emulation)

討論如何整合詮釋（Interpretation）與二進位翻譯（Binary Translation）來達成高效能。

- **效能權衡 (Tradeoff)**：詮釋執行啟動快但穩態效能低；二進位翻譯啟動慢（需轉換成本）但後續執行速度極快。
- **階段式模擬 (Staged Emulation)**：
  1. 程式啟動時先以**詮釋模式**執行，並進行剖析（Profiling）。
  2. 識別出執行頻率超過閾值的「熱點（Hotspots）」區塊。
  3. 將熱點翻譯為目標碼並存入**代碼快取（Code Cache）**。
- **翻譯單位**：可從單純的「基本區塊（Basic Blocks）」延伸到更大的「超區塊（Superblocks）」，以提供更多最佳化空間。

3.7 作業系統模擬 (Operating System Emulation)

在 ABI 層級，虛擬機必須模擬客體系統呼叫（System Calls）的語義。

- **相同 OS 案例**：當客體與主體 OS 相同時，只需透過「包裝程式（Wrapper/Jacket Code）」進行參數轉換（Copy/Convert），然後呼叫主體系統呼叫即可。
- **不同 OS 案例**：難度最高，必須完整模擬客體系統呼叫的語義並映射至主體 OS 的操作。
- **回呼函數 (Callbacks)**：處理如 Windows 事件驅動機制時，Runtime 必須攔截 OS 的發送常式（Dispatch Routine），確保在執行應用程式的回呼處理程式時仍能維持虛擬機的控制權。